CS418 HW7

Name: Evan Litzer
Date: 3/26/2026

5.6:

Inserting Point: $p = (p_x, p_y)$

1. Search where p belongs in the main tree
   - Start at the root of the main BST, and follow the search path using p_x until a null spot is reached where p should be inserted. BST is ordered by x coordinate ascending
2. Record every node on the search path
   - Keep the nodes from root down to the insertion position
   - For every node v, subtree set $P(v)$ gains the new point p. $T_{assoc}(v)$ must also gain p. Point is stored in associated structure along root to leaf path
3. Insert p as a new leaf in the main tree
   - BST insertion by x-coordinate.
4. Update all associated structures on the recorded path
   - For each node v on the path from root to new leaf:
   - Insert p into $T_{assoc}(v)$, the y-ordered BST for $P(v)$
   - After insertion, $P(V)$ changes to $P(v) \cup \{p\}$ for those nodes.
5. Only the associated structures change, as no other subtree has gained p in the main tree.

Deleting Point: $p = (p_x, p_y)$

1. Search for p in the main tree
   - Use the main BST on the x-coordinate to find the node storing p.
   - Record the root to node path while searching.
2. Remove p from associated structure on the path
   - Before/during deletion, delete p from $T_{assoc}(v)$ for every node v on the recorded path
   - After deleting p, each of the subtree sets lose the point, and y-structures must lose it too.
3. Delete p from the main tree
   - Do BST deletion in the x-tree
   - If p is a leaf then remove it
   - If p has one child, splice the child up

- If it has two children, replace with predecessor/successor. Then delete moved node in BST way

4. Two-child case
   - When deleting by replacing p with successor or predecessor q, then need to remove p and move and remove q from old place
   - Associated structures must stay aligned with subtree point sets
   - Delete p from all associated structures on search path to p
   - Carry out standard BST delete
   - If q node is removed from a different path, also update associated structures along path to the removed node so the stored subtree sets remain right.

Runtime:

One update $= O(h)$ main tree work and $O(h * h')$ associated tree work

$= O(h^2)$

$= O(\log^2 n)$ if the tree stays balanced.

5.7:

- 2D range tree query will visit $O(\log n)$ canonical nodes v in the main tree.
- These nodes come from sibling subtree along two root-to-leaf search paths.
- At depth i, a subtree has size $n_v = O(n/2^i)$, so:

$$\log n_v \leq \log(n/2^i) = \log n - i$$

Therefore:

$$\sum_v \log n_v \leq 2 \sum_{i=0,\, O(\log n)} (\log n - i)$$

This is an arithmetic series with $O(\log n)$ terms, each with size that is at most $O(\log n)$.

$$2 \sum_{i=0,\, O(\log n)} (\log n - i) = O(\log^2 n)$$

Canonical subsets are disjoint, so $\sum_v k_v = k$. So:

$$\sum_v \Theta(\log n_v + k_v) = \Theta(\log^2 n + k)$$

Therefore the analysis will only improve constants and not order of growth

5.9A:

- Query rectangle $[a : a] \times [b \times b]$, where is single point (a, b) stored.
- For kd-tree, range searching recurses into a child if the query rectangle intersects that child's region.
- Search algorithm is based on checking whether query rectangle intersects child region and recurses only if it does intersect.
- Query rectangle is one point though.
- A point can lie in at most one child region at each split, unless degenerate case where point on split line, assume it does not.
- At every internal node, follow only one child
- Kd-tree is build by median splits, height is $O(\log n)$
- Every level does $O(1)$ work
- Only one subtree is explored per level
- Height is $O(\log n)$

Therefore, membership by degenerate range query on kd-tree is $O(\log n)$

5.9B:

- For 2D range tree, standard query bound is $O(\log^2 n + k)$
- Because we find first $O(\log n)$ canonical subsets in the main x-tree, and then query associated y-structures for each of them

For special query $[a : a] \times [b : b]$ it is not needed though

- A point (a, b) is in the set iff
- There is a leaf in the main tree with $x = a$ and that point has $y = b$

Since main tree is balanced BST, can do BST search for a as x-coordinates are ordered.

- This takes $O(\log n)$ time

If no leaf exists, then it is not in the set.

If it does exist, inspect that stored point and check if its y-coordinate equals $b \to O(1)$

- Main tree of 2D range tree is a balanced BST on x-coordinate, with points at leaves

So time is $O(\log n)$

- Assume no two points have same x or y coordinates, there is at most one point with $x = a$. So searching main tree by x is enough to find the point. So not $O(\log^2 n)$ but rather $O(\log n)$

6.2:

Example: $s_i = [(-i, n-i+1), (i, n-i+1)]$ with $i = 1, 2, \ldots, n$

Segments are all horizontal, pairwise disjoint, and get longer as they go downward

Insert them in order $s_1, s_2, \ldots, s_n$

- Shortest one from the top to the bottom longest one

After $s_1, \ldots, s_{i-1}$ have been inserted, their endpoints have x-coordinates $-1, -2, \ldots, -(i-1), 1, 2, \ldots, i-1$

In trapezoidal map, these endpoints will create vertical walls/extensions

Since s_i runs from $x = -i$ to $x = i$, it crosses all those vertical walls, so s_i passes through

$$2(i-1) + 1 = 2i - 1$$

trapezoids

- If new segment intersects trapezoids $\Delta_0, \ldots, \Delta_k$:
- Then Δ_0 is replaced by an x node and a y node.
- Δ_k is replaced by an x node and a y node
- Each middle trapezoid is replaced by one y node
- Inserting s_i creates $\Theta(i)$ new internal nodes in the search DAG

Therefore total size is

$$\sum_{i=1,\, n} \Theta(i) = \Theta(n^2)$$

- For query point below all segments and near the center, the search follows a path that tests all segments $s_1, \ldots, s_n$ ins sequence.
- This gives worst case query time $\Theta(n)$.
- The construction algorithm updates search structure by replacing every trapezoid intersected by new segment, so this insertion order can force quadratic size

6.7:

If the center p is given:

- Star shaped polygon can be split into fan of triangles by drawing segments from p to every polygon vertex.
- Since every segment pv_i stays inside P, decomposition of P into n triangles around p.
- Simple polygon admits triangulation, and given star-center gives you fan from p.

Let vertices be $v_1, v_2, \ldots, v_n$ in boundary order.

Define the triangles $\Delta_i = (p, v_i, v_{i+1})$ for $i = 1 \ldots n$

Query Algorithm: Test whether $q \in P$

Find which wedge of the fan could contain q:

- Examine rays from p through the vertices v_i
- Vertices are already given in cyclic order, rays are also in cyclic order around p
- So use binary search for two consecutive rays pv_i and pv_{i+1} so q lies angularly between them
- $= O(\log n)$ time
- Standard orientation test suffices in that q is to the left or right of pv_i and compare against middle ray in the binary search

Test membership in the one candidate triangle

- Once wedge is found, only possible triangle is $\Delta_i = (p, v_i, v_{i+1})$
- Now test whether q lies inside the triangle using three orientation tests, which takes $O(1)$ time

If $q \in \Delta$, then $q \in p$. Otherwise, $q \notin p$

Total query time: $O(\log n) + O(1) = O(\log n)$

- P sees every point of the polygon, every segment pv_i lies inside P, so polygon is partitioned into fan triangles (p, v_i, v_{i+1})
- Therefore every point of P lies in one of these fan triangles, and checking candidates is enough

If p is not given:

First need to compute some point in the kernel of P

- Point is in the kernel exactly when can see the whole polygon, so kernel is intersection of planes determined by polygon edges.
- Half-plane intersection is a convex region

So:

- For each directed edge of P, form the interior half-plane
- Intersect all these half-planes
- If the intersection is nonempty, pick any point p in it

This will give a valid star center point p

Then use same methodology as previous part of this problem, where fan triangulation from that p, and answer queries.

Time remains $O(\log n)$